

对象序列化之一



对象状态保存与序列化

北京理工大学计算机学院
金旭亮

实际场景

当你用Word编辑一个文档时，突然断电，等电来了，你重新打开Word，你会发现Word会提示你：**有可以恢复的备份文档。**



如果要由你实现这种“定时备份”与“灾难恢复”的功能，你会怎么做呢？

“数据定时备份”技术解决方案



将文档对象保存到一个特殊文件中，需要时再从中还原。



这里可以派上用场的技术之一，就是“对象序列化”。



所谓“对象序列化”，其实就将某个时刻对象的状态（即对象所有字段当前值的集合）写入到一个“流（最常见的就是文件流）”中，在需要时再从流中读取数据重建对象在指定时刻的状态。

理解重要概念—对象状态



所谓“对象状态”，就是在特定时刻，内存中特定对象（所有）字段当前值的集合。

对象状态，是时间的单值函数。

$f(\text{时间}) \rightarrow \text{对象状态}$

对象序列化，其实就是将特定时刻对象状态中各个字段的当前值，以事先约定好的格式，写入到流中，这里“流”，可以是内存流或文件流。

对象的保存与重建的基本方法

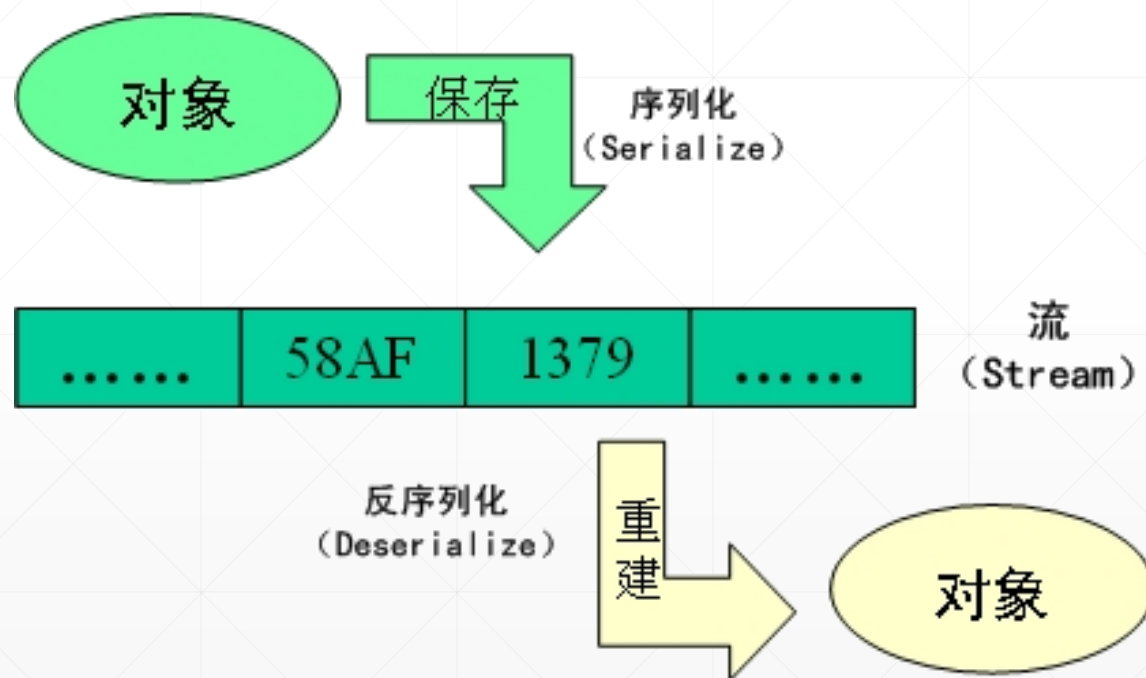
Java使用“对象序列化”的方法实现了在流中保存与重建对象的功能。

■ 序列化（保存）步骤：

1. 保存每个对象的类型
2. 保存对象每个属性的值

■ 反序列化（读取）步骤：

1. 读取对象类型
2. 创建一个该类型的空白对象
3. 用存储在流中的数据填充它



创建ObjectOutputStream实例以序列化对象



JDK 提供了 `ObjectOutputStream` 和 `ObjectInputStream` 两个类来实现对象序列化和反序列化的功能。

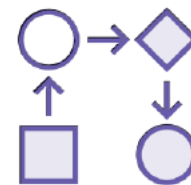


`ObjectOutputStream` 负责将对象状态保存到文件中，但它自己不负责存取文件，而是将这个工作委托给另一个流组件——`FileOutputStream` 来完成。

```
ObjectOutputStream out = new ObjectOutputStream(  
    new FileOutputStream("文件名"));
```

`out` 对象现在可以用于保存对象状态……

对象序列化实现代码



- 1 创建对象并保存其状态（即对象序列化）

```
MyClass Obj = new ... ;  
out.writeObject(obj);
```

- 2 从流中读取数值，重建对象并恢复其状态（即对象反序列化）

```
ObjectInputStream in =  
    new ObjectInputStream(  
        new FileInputStream("文件名"));  
MyClass obj = (MyClass) in.readObject();
```

ReadWritePerson.java示例



此示例将一个Person对象保存到一个 “person.data” 文件中，然后再从中读出来。

可以被序列化的类必须实现一个空的接口：**Serializable**，实现了这个接口的类的实例可以被序列化。



- 序列化和反序列化的成功，都要求应用程序能访问到类的 .class 文件。
- 当反序列化时，注意不会调用用户定义的构造函数。

示例分析-1

对象序列化过程:

```
private static void writePerson() {  
    //创建一个ObjectOutputStream输出流  
    try (ObjectOutputStream oos = new ObjectOutputStream(  
        new FileOutputStream(objDataFileName))) {  
        Person person = new Person( name: "张三", age: 30);  
        //将person对象写入输出流  
        oos.writeObject(person);  
        System.out.println("对象: " + person + "已写入到文件"  
            + objDataFileName + "中! ");  
    } catch (IOException ex) {  
        ex.printStackTrace();  
    }  
}
```

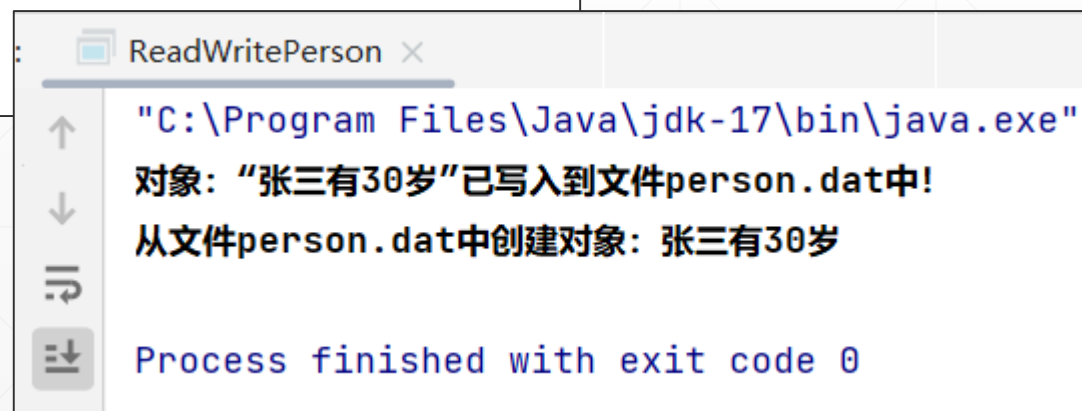
```
public class Person  
    implements java.io.Serializable {  
    4 usages  
    private String name;  
    4 usages  
    private int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    "getter and setter"  
  
    @Override  
    public String toString() {  
        return name + "有" + age + "岁";  
    }  
}
```

示例分析-2

对象反序列化过程

```
private static void readPerson() {  
    //创建一个ObjectInputStream输出流  
    try (ObjectInputStream ois = new ObjectInputStream(  
        new FileInputStream(objDataFileName))) {  
        //从输入流中读取一个Java对象，并将其强制类型转换为Person类  
        Person person = (Person) ois.readObject();  
        System.out.println("从文件" + objDataFileName + "中创建对象: " + person);  
    } catch (IOException | ClassNotFoundException e) {  
        throw new RuntimeException(e);  
    }  
}
```

生成的数据文件：person.dat，是一个二进制文件，而非文本文件。



```
ReadWritePerson x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
对象: "张三有30岁"已写入到文件person.dat中!  
从文件person.dat中创建对象: 张三有30岁  
Process finished with exit code 0
```